Cross-Corpus Evaluation of 5G Flow Intrusion Detectors: Label Faults, Transfer, and Alert Burden

ADEEL AHMAD[1], ARSHAD ALI[1], ERAJ KHAN[2], GAHANGIR HOSSAIN[3], AND ALI AKARMA[1]
[1]Faculty of Computer and Information Systems, Islamic University of Madinah, Al Madinah Al Munawarah, Saudi Arabia
[2]Faculty of Computer Information Science, Higher Colleges Of Technology, Al Mizn- Baniyas North- Abu Dhabi, United Arab Emirates (e-mail: ekhan@hct.ac.ae)
[3]Anuradha and Vikas Sinha Department of Data Science, University of North Texas, Denton, TX, USA, 76203-5017

Corresponding author: Eraj Khan (e-mail: ekhan@hct.ac.ae).

ABSTRACT Intrusion detectors for 5G networks are often reported with more than 99% accuracy on a random split of one corpus. We measure what such scores predict on traffic from another network, for six classifiers on three public corpora: NetsLab-5GORAN-IDD, 5G-NIDD, and a 5G core testbed. Only the first comes from an O-RAN testbed, so the cross-corpus results concern flow-level detectors on 5G traffic; radio-telemetry detectors are evaluated within one corpus, their latency is measured separately in a near-real-time RIC loop, and no detector is evaluated on every criterion. Four results stand out. First, 59% of the benign flows of 5G-NIDD are another base station's UDP flood: in that session, the raw capture of station 1 contains every packet of station 2's capture, and a published 99.9% accuracy rests on two record-position fields. Second, once these labels and a byte-accounting difference between flow exporters are corrected, tree-based detectors trained on NetsLab-5GORAN-IDD score as well on 5G-NIDD as on held-out source data, yet 0.12 to 0.19 below references trained on 5G-NIDD and tested on the same flows. Third, at an attack prevalence of 0.002 and alert bounds that we set ourselves, no detector passes our two-term deployability test: verdicts are fail or inconclusive, because a threshold fixed on source data lands differently on the target in each training run. One labeled target capture helps more than three adaptation methods. Fourth, on the radio layer, a random split raised macro-F_1 by 0.13, but not significantly. Random-split scores did not predict cross-corpus performance.

INDEX TERMS Dataset shift, evaluation methodology, false positive rate, intrusion detection, label quality, network flows, O-RAN, 5G.

I. INTRODUCTION

Open Radio Access Network (O-RAN) opens the base station to third-party control applications that run in a RAN Intelligent Controller (RIC) [1]–[3]. A detection application (xApp) in the near-real-time RIC could look for attacks close to where they enter the network [1], [4], for example floods from compromised devices [5], [6], and detectors in or beside the O-RAN control loop have been built, each in its own testbed [7], [8]. Most machine-learning detectors for 5G, however, are trained and tested on flow records from one testbed.

Their reported scores are often very high. On 5G-NIDD, one of our corpora, the dataset authors report binary accuracies of 99.85% to 99.95% for four of five classifiers on a random split [9], and later work reports a weighted F_1 of up to 99.95% [10]. A random split measures how well a detector separates the records of one corpus. Deployment depends on other properties: whether the detector works on traffic from a network that contributed no training data, where intrusion detectors are known to transfer poorly [11], [12], and how many false alerts it raises at production flow rates and a realistic attack base rate [13], [14]. We ask what a held-out score on one corpus predicts about these two properties, and, unable to observe a deployment, study generalization through transfer to independently collected corpora.

Scope. We use three public corpora: NetsLab-5GORAN-IDD, recorded on an O-RAN testbed with flow records and radio telemetry [15]; 5G-NIDD, recorded on a 5G test network [16]; and the flow records of a 5G core testbed [17]. Only the first is an O-RAN corpus, and only its flow records have counterparts in the other two. Our cross-corpus results therefore concern flow-level detectors on 5G traffic. The radio-telemetry detectors, the O-RAN-specific part, are evaluated within NetsLab-5GORAN-IDD only, and their decision

latency is measured separately, in a near-real-time RIC loop with an emulated E2 node. No single detector is evaluated on generalization, alert burden, and latency together, and we make no claim that any detector is deployable in O-RAN.

Transfer between NetsLab-5GORAN-IDD and 5G-NIDD has been measured before [18], and so has inference time inside a near-real-time RIC [19]. We offer neither a new detector nor the finding that transfer hurts. Our contributions are:

- An audit of the 5G-NIDD labels (Section V-A). In one capture session, the file of one base station contains every packet recorded at the other, and the other station's flood is labeled benign there; these copies are 59% of the benign class. We show how a published 99.9% accuracy depends on two record-position fields, and release corrected labels.
- A cross-corpus measurement of six flow detectors with matched references (Sections V-B and V-C). After correcting the labels and a byte-accounting difference between flow exporters, we measure how far transfer falls below references trained on the target and tested on the same flows, what the gap consists of, and how much three adaptation methods and a few labeled target captures recover.
- Alert burden at a declared base rate, and a two-term deployability test whose terms treat uncertainty the same way and can be inconclusive (Sections III and V-D).

We also report how the radio-layer score depends on the split protocol, a result that is not statistically significant (Section V-E), and the latency of the RIC loop for window-level detectors (Section V-F).

II. BACKGROUND AND RELATED WORK

A. DETECTION IN THE NEAR-REAL-TIME RIC

An xApp talks to E2 nodes through E2 service models (E2SM): E2SM-KPM delivers periodic key performance measurements (KPM), and E2SM-RC exposes control actions [1]. Platforms such as OpenRAN Gym [20], SCOPE [21], ColO-RAN [22], and FlexRIC [23], with open radio stacks [24], [25], made experimental xApps practical. E2 carries aggregated measurements, not packets, so a flow-level detector needs a data path of its own (Fig. 1). The RIC and its xApps are also a new attack surface [26]–[28], and security mechanisms for the O-RAN interfaces have been built and evaluated [29]. On the detection side, 5G-Spector offers layer-3 attack detection as an O-RAN service [7], DetRAN detects attacks from cross-layer data in real time [8], and Fard et al. fuse radio telemetry and flow records of NetsLab-5GORAN-IDD [30]. Each is evaluated in the setting it was built for; our question is what such an evaluation predicts about another network.

B. INTRUSION DETECTION CORPORA AND THEIR FAULTS

Public intrusion corpora are plentiful for IoT traffic [31]–[35] and scarce for cellular networks [15]–[17], [36]. Standardized NetFlow feature sets made cross-corpus experiments easier [37], [38]. The labels and flow records of a corpus can themselves be wrong: Engelen et al. documented labeling and flow-construction faults in CIC-IDS2017 [39], and Flood et al. found design flaws across NIDS benchmarks that let a classifier score well without learning the attack [40].

C. EVALUATION PRACTICE, DATASET SHIFT, AND ADAPTATION

Reported results tend to overstate what a detector can do, for known reasons: closed-world evaluation [14], neglect of the base rate [13], sampling bias and data leakage [41], [42], and temporal or spatial bias in the split [43]. The gap between reported and operational effectiveness has been traced across security domains [44] and in industrial intrusion detection [45], and cross-dataset generalization has repeatedly proved poor [11], [12], [46]. Closest to our work, Abraheem and Edhirig transfer XGBoost between NetsLab-5GORAN-IDD and 5G-NIDD with 15 harmonized features and report zero-shot balanced accuracies per attack family of 0.50 to 0.56 in our direction [18]. We add input-blind baselines, matched in-target references, an audit of the target labels and byte accounting, adaptation, and the operational criterion.

Three families of methods address a shift between a labeled source and an unlabeled target: reweighting source samples by an estimate of the density ratio [47], [48], aligning feature statistics, such as the covariance alignment of CORAL [49], and learning a representation in which the two domains cannot be told apart, as domain-adversarial training does [50]. We test one method of each family. Classifiers compared under repeated resampling are tested as Nadeau and Bengio recommend [51].

III. EVALUATION CRITERIA

Fig. 1 shows the setting: a detection xApp in the near-real-time RIC, a radio-telemetry lane fed by E2SM-KPM, and a flow lane fed by a user-plane exporter that E2 does not provide. The adversary controls some user devices and uses them for flooding, scanning, brute-force login, web attacks, or, in one corpus, an attack on the control plane of the core network, the attack categories in our corpora. It does not control the RIC, the E2 interface, or the xApp. The figure also marks which criterion we measured on which lane.

A. GENERALIZATION ACROSS CORPORA

Let $\mathcal{D}_A$ and $\mathcal{D}_B$ be two independently collected corpora in a shared feature space. We measure a model f_θ trained on $\mathcal{D}_A$ by how far its score on $\mathcal{D}_B$ falls below its held-out score on $\mathcal{D}_A$. The corpora differ in attack prevalence (Table 1), and macro-F_1 changes with prevalence even for a classifier that ignores its input, so we use balanced accuracy (BA), the mean of the true positive and true negative rates, which prevalence does not affect:

$$\Delta_{\mathrm{BA}} = \mathrm{BA}\left(f_\theta; \mathcal{D}_A^{\mathrm{test}}\right) - \mathrm{BA}(f_\theta; \mathcal{D}_B)\,. \tag{1}$$

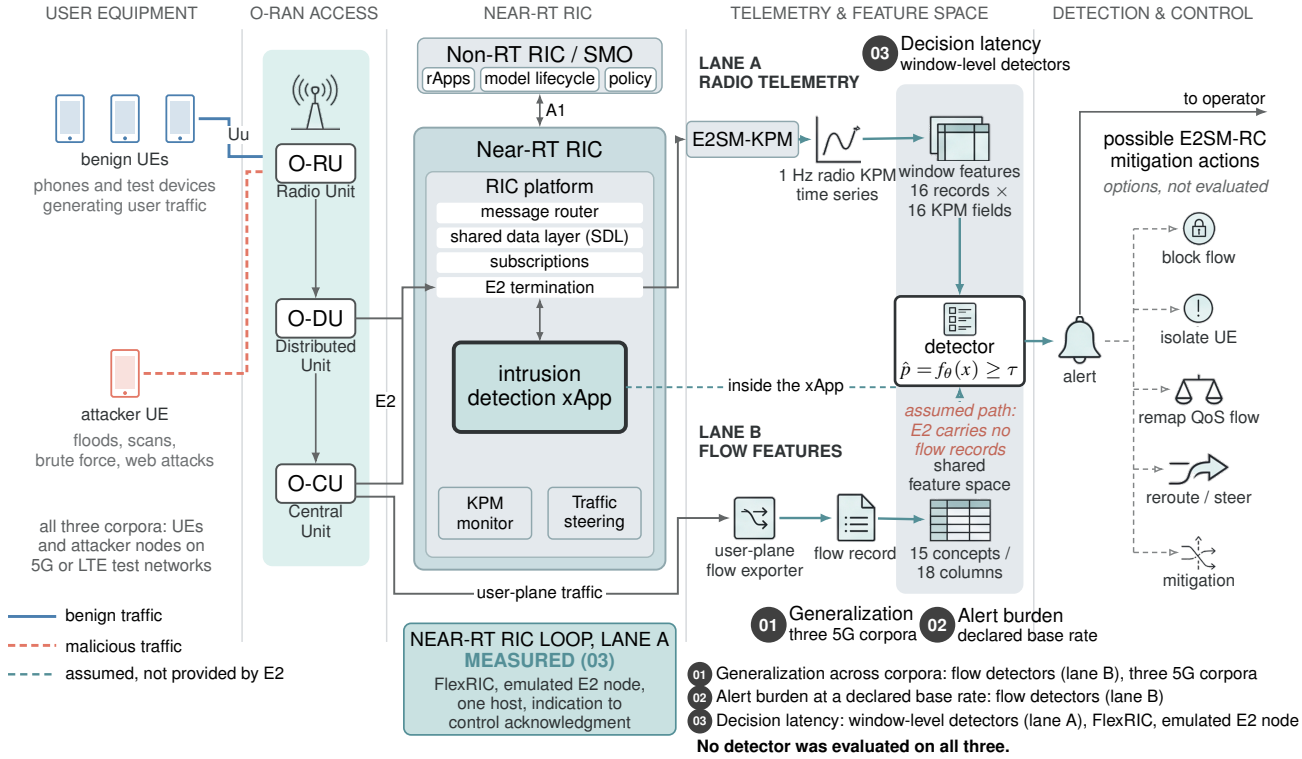


FIGURE 1. Reference O-RAN deployment, the two detection lanes, and what we measured on each. Lane A: window features from E2SM-KPM reports. Lane B: flow records from a user-plane exporter; O-RAN defines no path that carries them to an xApp, so the dashed arrow is an assumption. Generalization across corpora (01) and alert burden (02) are measured for flow detectors (lane B) on three 5G corpora; decision latency (03) is measured for window-level detectors (lane A), from indication to control acknowledgment, through FlexRIC with an emulated E2 node on one host. No detector was evaluated on all three. The mitigation actions on the right are not evaluated.

Because the source score is itself optimistic, we also compare transfer with references trained on $\mathcal{D}_B$ and tested on the same $\mathcal{D}_B$ flows (Section IV-D).

B. ALERT BURDEN UNDER A DECLARED BASE RATE

Let λ_b be the benign flow arrival rate. For a decision threshold τ with false positive rate $\mathrm{FPR}(\tau)$ and recall $R(\tau)$, false alerts arrive at $A(\tau) = \lambda_b \mathrm{FPR}(\tau)$, and at an operational attack prevalence π the fraction of alerts that are real is

$$\mathrm{PPV}(\tau) = \frac{\pi R(\tau)}{\pi R(\tau) + (1-\pi)\,\mathrm{FPR}(\tau)}. \tag{2}$$

We call this the operational precision; the corpus precision is the precision at the corpus's own prevalence. The corpora give neither π nor λ_b, so we declare both (Section IV-F); A grows linearly with λ_b, so alert counts at another rate follow from the false positive rates we report.

C. A TWO-TERM DEPLOYABILITY TEST

We combine the two criteria into a test for flow detectors. Each term has three outcomes, and both use the same corrected 95% intervals over the splits (Section IV-E):

- *Generalization* passes if the upper bound of Δ_{BA} is at most δ, fails if its lower bound exceeds δ, and is inconclusive otherwise.
- *Alert burden* is evaluated at a threshold fixed without target labels: for each split, the highest threshold that keeps recall on the held-out source data at $r_{\min}$ or more. At that threshold on the target, it passes if the intervals of recall, false alerts, and operational precision all lie on the passing side of $r_{\min}$, $A_{\max}$, and ρ; it fails if any interval lies entirely on the failing side; and it is inconclusive otherwise.

A detector passes if both terms pass and fails if either term fails. A threshold searched on the target cannot be chosen in a deployment; we report the best such threshold only as an optimistic bound. No O-RAN specification defines these bounds, so we chose them ourselves: $\delta = 0.10$, $r_{\min} = 0.5$, $\rho = 0.1$ at $\pi = 0.002$, and $A_{\max} = 200$ false alerts per hour at $\lambda_b = 240{,}000$ benign flows per hour. The last bound is a false positive rate of at most 8.3×10^{-4}. The test is a reporting instrument, not a requirement; Section V-D shows how the verdict responds when the bounds change.

D. LATENCY, REPORTED SEPARATELY

The decision latency of an xApp decomposes as

$$L = t_{\mathrm{ind}} + t_{\mathrm{feat}} + t_{\mathrm{inf}} + t_{\mathrm{act}} + t_q, \tag{3}$$

the delivery of the indication to the xApp, feature construction, inference, the control action up to its acknowledgment, and queueing inside the xApp. The O-RAN E2 specification

TABLE 1. Corpora. $\mathcal{D}_A$ supplies training and in-distribution testing on both layers; $\mathcal{D}_B$ and $\mathcal{D}_C$ are transfer targets. $\mathcal{D}_B$ also has in-target references and is the source for the reverse direction.

	$\mathcal{D}_A$ flows	$\mathcal{D}_A$ radio	$\mathcal{D}_B$	$\mathcal{D}_C$
Unit	flow	16 s UE window	flow	flow
Source	Zeek	KPM, 1 Hz	Argus	NFStream
Samples	1 640 182	2 808	1 215 889	39 425
Features	18	32	18	18
Attack prevalence	94.63%	76.46%	60.71%	2.13%
Attack categories	5	5	2	3
Groups	318 hosts	30 sess. (42 runs)	20 files	3 files
Bytes counted	IP	–	Eth.*	Eth.+GTP-U*
Role	train, test	train, test	target, ref.	target

*Ethernet header (and GTP-U) removed per packet (Section IV-B).

allows control loops of 10 ms to 1 s [52]. We measured all five terms for window-level detectors only, so latency is not part of the test; Section V-F reports it as a separate measurement.

IV. DATA AND METHODS

A. CORPORA

$\mathcal{D}_A$ is NetsLab-5GORAN-IDD [15], recorded on a physical OpenAirInterface O-RAN testbed, with benign traffic and five attack categories: denial of service (DoS), distributed denial of service (DDoS), probing, brute force, and web attacks. Its flow records were exported by Zeek at the central unit, and its radio telemetry was sampled at 1 Hz at the distributed unit; the two layers cannot be joined record by record from the published files. For the radio layer we cut the records of each UE into consecutive, non-overlapping windows of 16 records and summarize each window by the mean and standard deviation of 16 KPM fields. Gaps of more than five minutes in the timestamps split the radio layer into 30 capture sessions; a gap of 90 to 105 s recovers the 42 experiment runs described by Fard et al. [30], and every session is a union of whole runs. All records of a session or run carry the same category label.

$\mathcal{D}_B$ is 5G-NIDD [16], [36], recorded on a 5G test network with two base stations and exported by Argus, with two attack categories (DoS and probing). Its combined file has no addresses, ports, or timestamps, but it keeps the Argus record counter `Seq` and the file offset `Offset`, which restarts at every file boundary, so we recover the 20 capture files, ten per base station (files 1 to 10 from station 1, 11 to 20 from station 2). Both stations went through the same sequence of capture sessions: nine files each hold benign traffic and one attack type, and the tenth carries no attack label. A second release keeps every Argus field, addresses included, and the raw captures are public too.

$\mathcal{D}_C$, a second target at a third site, is the flow records of a 5G core testbed (Open5GS in Docker) [17]: three files, each with one attack (a SYN flood, an ICMP flood, and a PFCP session-deletion attack), 39,425 flows, 839 of them attacks. Its benign flows mix user traffic with core-network signaling. Table 1 summarizes the corpora.

TABLE 2. Shared flow feature space. Byte columns hold IP-layer bytes: for 5G-NIDD (Argus) and the 5G core testbed (NFStream), 14 bytes of Ethernet header per packet are removed, and for tunneled NFStream packets also 44 bytes of GTP-U encapsulation. The exporter-robust subset is the protocol one-hot, the three mean packet sizes, and the two direction ratios.

Concept	$\mathcal{D}_A$ (Zeek)	$\mathcal{D}_B$ (Argus)	Role
duration	duration	Dur	base
src_bytes	src_ip_bytes	SrcBytes − 14 SrcPkts	base
dst_bytes	dst_ip_bytes	DstBytes − 14 DstPkts	base
src_pkts	src_pkts	SrcPkts	base
dst_pkts	dst_pkts	DstPkts	base
tot_pkts	src_pkts + dst_pkts	TotPkts	base
tot_bytes	src + dst ip_bytes	src_bytes + dst_bytes	base
proto	proto	Proto	one-hot (4)
mean_pkt_size	tot_bytes / max(tot_pkts, 1)		derived
src_mean_pkt_size	src_bytes / max(src_pkts, 1)		derived
dst_mean_pkt_size	dst_bytes / max(dst_pkts, 1)		derived
bytes_per_s	tot_bytes / max(duration, 1 ms)		derived
pkts_per_s	tot_pkts / max(duration, 1 ms)		derived
src_byte_ratio	src_bytes / max(tot_bytes, 1)		derived
src_pkt_ratio	src_pkts / max(tot_pkts, 1)		derived

B. BYTE ACCOUNTING AND THE SHARED FEATURE SPACE

Zeek and Argus share no column names, so we define the flow feature space by meaning (Table 2): 15 concepts in 18 columns, namely seven base quantities (duration and packet and byte counts), a one-hot protocol with a fixed list of levels, and seven derived quantities computed the same way for every corpus.

The exporters count bytes differently, and the difference must be removed before any comparison. Zeek's `src_ip_bytes` counts IP packets, header included. Argus counts the Ethernet frame. In 5G-NIDD the smallest bytes per packet are 54 for TCP and 42 for UDP and ICMP, where Zeek, on $\mathcal{D}_A$ and on our own re-extraction of the 5G-NIDD captures, counts 40 and 28. The captures are Ethernet frames, and in four captures we checked, the frame length exceeds the IP length by exactly 14 bytes for 94.5% of the frames (most of the rest carry a VLAN tag), so Argus adds the 14-byte Ethernet header to every packet. NFStream, the exporter of $\mathcal{D}_C$, adds the same header and, for tunneled packets, 44 bytes of GTP-U encapsulation. We subtract these bytes per packet and direction for $\mathcal{D}_B$ and $\mathcal{D}_C$, so that every byte column holds IP-layer bytes, as Zeek's do. The correction matters most for small packets: a UDP flood packet counts 28 bytes after it and 42 before. All corpora get the same 1 ms floor on duration and $\log(1+x)$ transform for unbounded quantities; scaling is fitted on source training data alone, and no address, port, or timestamp is a feature. Projecting $\mathcal{D}_A$ onto the shared columns costs 0.005 to 0.082 of in-distribution balanced accuracy against all 46 non-port Zeek features.

Before computing any result with it, we fixed a subset of nine columns that we expected to depend less on the exporter, which we call exporter-robust: the protocol, the three mean packet sizes, and the two direction ratios. Where an exporter ends a flow moves duration, rates, and totals; per-packet means and direction ratios move less but are not invariant, since splitting a flow changes them.

C. DETECTORS AND ADAPTATION

Every protocol runs six supervised architectures: logistic regression (LR), a decision tree (DT) of depth 12, a random forest (RF) [53], XGBoost (XGB) [54], histogram gradient boosting (HGB) from scikit-learn [55], and a multilayer perceptron (MLP). Two input-blind baselines, a majority-class predictor and a stratified sampler of the training prior, can change between protocols only through class composition. All supervised models use balanced class weights (sample weights for the MLP, `scale_pos_weight` for XGB). We fixed the hyperparameters in advance, because tuning on held-out source data would reward source-specific structure, which the transfer experiments measure, and tuning on the target would leak target information into training: LR with `max_iter` 2000 on standardized inputs; RF with 200 trees and minimum leaf size 2; XGB with 300 trees of depth 6, $\eta = 0.1$, subsample 0.8, and the `hist` method; HGB with 300 iterations of depth 6 and $\eta = 0.1$; and an MLP with hidden layers of 128 and 64 units, trained with Adam and early stopping (patience 10) on standardized inputs. These are untuned shallow models; our conclusions concern them, not O-RAN detection in general.

Three adaptation methods, one per family of Section II, read unlabeled target features:

- *Importance weighting* (IW) [47], [48]: a logistic-regression domain classifier, trained on the source training flows against an equal number of target flows, estimates the density ratio, whose clipped (1st to 99th percentile) and normalized values weight the source flows of each of the six architectures.
- *CORAL* [49]: the source flows are re-colored to the covariance of the target (per-domain z-scores, identity-regularized covariances) before the six architectures are trained.
- *Domain-adversarial training* (DANN) [50]: a network learns features from which a label head classifies source flows while a domain head, whose gradient is reversed, cannot tell source from target (layers of 64 and 32 units, five passes over the source data, the schedule of Ganin et al. for the reversal weight, balanced class weights).

To ask how much labeled target data is needed, a label-efficient arm adds $k \in \{1, 2, 4, 8\}$ labeled 5G-NIDD capture files to training, four random draws per k, either as up to 60,000 flows next to the $\mathcal{D}_A$ training data or alone (up to 240,000 flows), and tests on the capture files not drawn.

D. EVALUATION PROTOCOLS AND REFERENCES

Source splits. On the flow layer the group key is the source address (318 groups). For each of 20 split seeds we draw a seeded sample of 300,000 flows from $\mathcal{D}_A$ and hold out whole source addresses until at least 20% of the flows are held out, keeping the attack prevalence of the held-out part close to the overall one. Because a few attacker addresses carry most of the flows (3 to 5 addresses carry 90% of each attack category), the held-out part contains 2 to 149 addresses and 29% to 52% of the flows. Every attack category appears in training in all 20 splits, with at least 29% of its flows. Of the 318 addresses, 259 send attack flows; the repository lists the attacker addresses per category and the coverage of every split. We test transfer on every flow of each target. The sample size of 300,000 flows was fixed before the first run, for runtime: the MLP takes several minutes per split; a check with 150,000, 600,000, and all 1,640,182 source flows (five splits, five architectures) changed clean target balanced accuracy by at most 0.08, in no consistent direction (Section V-C). On the radio layer, three protocols each hold out about 20% of the windows: a random split, a group-disjoint split, and a category-stratified group-disjoint split that keeps groups of every attack category on both sides, with sessions or runs as the group.

Label views of $\mathcal{D}_B$. Section V-A shows that 281,529 benign-labeled flows of $\mathcal{D}_B$ are copies of attack traffic. We evaluate on all flows with the published labels, on the flows without the copies (“clean”), and with the copies relabeled as attack (“relabeled”), which keeps the traffic a deployed detector would see.

In-target references. To separate the cost of transfer from the difficulty of $\mathcal{D}_B$, we train the same architectures on $\mathcal{D}_B$ and test them on $\mathcal{D}_B$, with the copies removed from training (clean) or relabeled as attack (relabeled), always on 240,000 training flows, the size of the transfer training sets. A random split of a 300,000-flow sample (20 seeds) is the optimistic reference. Leave-one-capture-file-out holds out each of the 20 files once, so both UDP-flood files are held out, and its pooled predictions cover every flow of $\mathcal{D}_B$ exactly once, which makes it comparable with transfer flow for flow. Training on one base station and testing on the other (ten training samples per direction) is the strictest reference. The reverse direction trains on $\mathcal{D}_B$, split by capture file, and tests on $\mathcal{D}_A$ and $\mathcal{D}_C$.

Apart from the references, the adaptation methods, the label-efficient arm, and the target-searched thresholds reported as bounds, no transform, threshold, or model choice touches a target, and every access to it is logged.

E. STATISTICAL ANALYSIS

We estimate uncertainty over 20 splits. The splits resample one population of source addresses or radio sessions, so their training and test sets overlap, and a paired t-test over split means understates the variance. Our intervals are therefore Nadeau–Bengio corrected [51]: the variance over J splits is scaled by $1/J + n_{\text{test}}/n_{\text{train}}$. Because samples within an address or session are correlated, we compute this ratio from groups, not from flows or windows, as the total of held-out groups over the total of training groups across the 20 splits; a mean of per-split ratios would be dominated by the splits that hold out 149 addresses. The ratio is 0.65 for the flow splits (0.58 in flows), and 0.42 for both radio groupings. The target is fixed, so an interval covers the variation from resampling the source, not that across capture files or target corpora. We also tried a cluster bootstrap [56] over the capture files of $\mathcal{D}_B$, but with 20 purpose-built files of very different content, one of which holds most benign errors, its distribution of

balanced accuracy and false positive rate was strongly shifted, and resampling source addresses inside a split often left a single class, because some splits hold out only two to nine addresses; we report it only in the repository. Both terms of the deployability test use the same corrected intervals: the generalization term on the per-split Δ_{BA}, the alert term on the per-split recall and false positive rate at each split's threshold, from which operational precision and false alerts follow. Holm correction [57] is applied within each family of one metric and one comparison across the six architectures; bracketed intervals in the tables are unadjusted, and significance marks are Holm-adjusted. Operating points come from confusion counts pooled over splits, because a mean of per-split precision rewards splits without a false positive.

F. EXPERIMENTAL SETUP

All measurements ran on one workstation (13th Gen Intel Core i9-13900H, 14 cores, 20 threads, 47.6 GB of memory, Windows 11 Pro) with Python 3.11.9, scikit-learn 1.8.0, XGBoost 3.2.0, NumPy 2.3.5, and ONNX Runtime 1.30.0; the full package list ships with the results. We declare an attack prevalence of $\pi = 0.002$ and a benign rate of $\lambda_b = 240{,}000$ flows per hour. Neither comes from an operator; this rate of about 67 flows per second is below the several hundred per second that Jin et al. extrapolate from measured radio-bearer arrivals at a heavily loaded base station of a large LTE network [58]. For the latency loop we built FlexRIC [23] v2.0.0 in WSL2 on the same machine (Section V-F).

V. RESULTS

A. THE 5G-NIDD LABELS AND A PUBLISHED RESULT

Label conflicts. For half of $\mathcal{D}_B$ the labels do not follow from the flow records: 52% of its flows match a flow with the opposite label in every field except the row index, `Seq`, and `Offset` (Table 3). All of them sit in the two UDP-flood files, file 5 of station 1 and file 15 of station 2. Each of the 281,529 flows labeled UDP flood in file 15 has a copy labeled benign in file 5 that agrees in every content field. In the release that keeps every field, these benign copies are all UDP flows from 10.155.15.7 to 10.41.150.68, the host pair of station 2's flood, whereas the attack labeled in file 5 comes from another host, 10.155.15.4. The copies make up 59% of the benign class.[1]

The raw captures. The two UDP-flood captures show where the copies come from (Fig. 2). Every one of the 656,180 IPv4 packets of station 2's capture also occurs in station 1's capture, with the same timestamp to the microsecond and the same addresses, IP identification, and length; they make up 54% of station 1's file, and both captures start at the same microsecond. In the other nine sessions, the two stations' captures share no packet and start 0.007 to 0.64 s apart, as two separate recordings would. The dataset description makes such an overlap possible: both base stations connect over S1-U to one switch, which also connects the attackers to the target server [9], and in this session both attackers flooded the same server, overlapping for about five minutes, as the published schedule describes. Station 2's capture contains no packet of station 1's attacker, which explains the asymmetry: file 15 holds no copies of station 1's flood because station 2's file never contained that traffic. How station 1's file came to contain station 2's traffic is not documented; the observations fit a recording on a link that carries both stations' traffic, in this one session. Since the copies are station 2's flood packets, we treat them as attack traffic with a wrong label. We change no published label. We report results on all flows, without the copies (clean), and with the copies relabeled as attack, and we release the 281,529 corrected rows with our code. We reported the finding and the evidence to the dataset authors on [DATE].

[1]The authors' encoded release counts 281,525 copies, because it fills missing values of four of them differently.

TABLE 3. Label conflicts in 5G-NIDD. Content: all fields of the published encoding except row index, `Seq`, `Offset`, and the labels. Ceiling: balanced accuracy of the best lookup on distinct records, in sample.

Features	Cols	Records	% in conflicting records			BA
			All	Benign	Attack	ceiling
Content fields	90	374 946	52.0	58.9	47.5	0.766
Shared space	18	246 281	62.4	59.7	64.1	0.753
Content + `Seq`, `Offset`	92	1 215 869	0.0	0.0	0.0	1.000

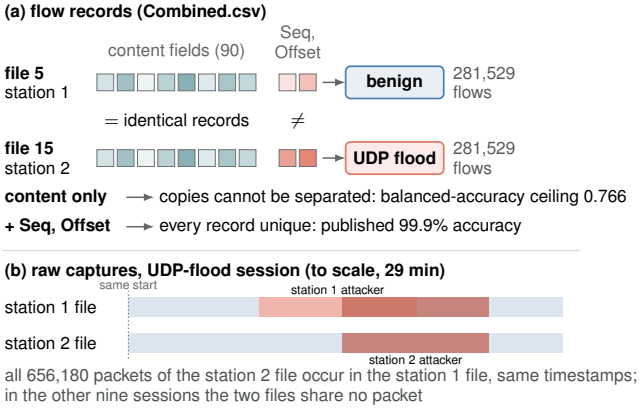


FIGURE 2. The label conflict in 5G-NIDD. Top: identical flow records occur in two capture files, labeled benign in station 1's file and UDP flood in station 2's (schematic tiles). They differ only in the row index and the record-position fields `Seq` and `Offset`, which make almost every record unique, account for the published 99.9%, and are never seen by a deployed detector. Bottom: in the raw captures of this session, station 1's file contains every packet of station 2's file with the same timestamp; in the other nine sessions the two files share no packet.

What the copies do to a classifier. Identical records under opposite labels cap any classifier that reads only record content: even on the flows it was built from, the best function of the 90 content fields reaches a balanced accuracy of 0.766, and of the 18 shared columns 0.753. With `Seq` and `Offset`, almost every record becomes unique and the ceiling rises to 1.000.

A published result under the same protocols. The dataset authors report binary accuracies of 99.85% to 99.95% on a random 70/30 split [9]. We rebuilt their pipeline (the published one-hot encoding, the ten features with the highest

ANOVA F-scores, z-score normalization, and their five classifiers with scikit-learn defaults, as tuned values are not reported) and obtain 99.87% to 99.96% for DT, RF, KNN, and the MLP (Table 4). Their two top features are `Seq` and `Offset`, which no deployed detector observes and which are exactly what separates the conflicting records. Replacing only these two fields with the next two in the ranking brings the accuracy of DT, RF, and the MLP down to 76.8% to 76.9% (balanced accuracy 0.704 to 0.706), yet RF still reaches 0.9999 on the flows without the copies. With capture files held out, the published features give balanced accuracies of 0.859 to 0.923 and, without the two fields, 0.799 to 0.800; accuracy falls to 1.6% on the split that trains on one UDP-flood file and tests on the other. Across base stations the published features give 0.762 to 0.895. A 99.9% accuracy on this corpus says little about a new capture file or base station, and nothing about a deployment, where record positions do not exist.

B. TRANSFER ACROSS CORPORA

Table 5 and Fig. 3 give the transfer from $\mathcal{D}_A$ to $\mathcal{D}_B$, with the byte correction, over 20 splits. On all flows, with the published labels, balanced accuracy falls from 0.81 to 0.90 on held-out source data to 0.52 to 0.63 on $\mathcal{D}_B$, a loss that is significant after Holm correction for five of the six architectures. Much of this loss is the label conflict: a detector that has learned to recognize a UDP flood on $\mathcal{D}_A$ also flags the benign-labeled copies in file 5.

On the clean flows, target balanced accuracy is 0.61 to 0.86. The four tree-based architectures score as well on $\mathcal{D}_B$ as on held-out $\mathcal{D}_A$: their target balanced accuracy is 0.78 to 0.86, and their loss against held-out source data is -0.01 to 0.04, none of it significant. LR and the MLP transfer worst (0.61 and 0.69), and only the loss of LR is significant after Holm correction. Relabeling the copies as attack, which keeps every flow of $\mathcal{D}_B$, gives the same picture (0.57 to 0.89). The label conflict reverses the ranking of the architectures: LR, which flags almost none of the copies, looks best on all flows and is worst on consistent labels. Without the byte correction, our first analysis of the same flows had found 0.74 to 0.78 for the five nonlinear architectures; with it, the tree-based models reach 0.78 to 0.86, while the MLP falls to 0.69.

The source score is not the right yardstick, however, because $\mathcal{D}_B$ is easier than held-out $\mathcal{D}_A$. Trained on $\mathcal{D}_B$ itself, the same architectures reach 0.98 to 0.99 on a random split and 0.96 to 0.98 when every capture file is held out once. On the same flows, transfer falls 0.12 to 0.35 short of this leave-one-capture-file-out reference (Table 5). The shortfall is significant after Holm correction for three of the six architectures; for the others the intervals are wide, because target balanced accuracy varies much between splits, which train on different source addresses. Trained on one base station and tested on the other, the reference still reaches 0.97 to 0.99 from station 1 to station 2 and 0.97 to 0.99 in the other direction, so a model trained on 5G-NIDD traffic carries over to the other base station far better than one trained on $\mathcal{D}_A$ carries over to 5G-NIDD.

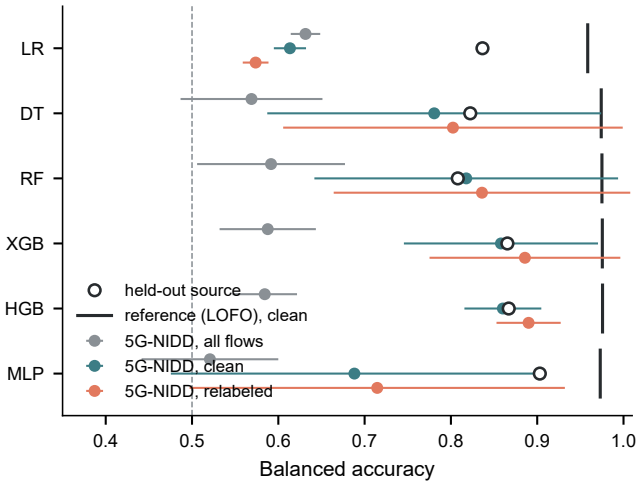


FIGURE 3. Balanced accuracy of detectors trained on $\mathcal{D}_A$, on held-out $\mathcal{D}_A$ (open circles) and on $\mathcal{D}_B$ (filled, with corrected 95% interval over 20 splits) for all flows, clean flows, and with the copies relabeled. Vertical bar: the reference trained on $\mathcal{D}_B$ with each capture file held out, on the clean flows. On consistent labels the tree-based models score as well on $\mathcal{D}_B$ as on held-out $\mathcal{D}_A$, but well below what $\mathcal{D}_B$ itself allows.

A third corpus. From $\mathcal{D}_A$ to $\mathcal{D}_C$ (Table 6, 20 splits), balanced accuracy is 0.60 to 0.74 and ROC-AUC 0.70 to 0.93; the loss against held-out source data is not significant for any architecture. Inside $\mathcal{D}_C$, the same architectures reach 0.987 to 0.990 on a random split and 0.660 to 0.826 when the capture file of one attack type is held out. Every architecture detects the SYN flood, which training contained (recall 1.00 for all six), whereas recall on the ICMP flood is 0.10 to 1.00 and on the PFCP attack, an attack type absent from $\mathcal{D}_A$, 0.00 to 0.99. Most errors come from the benign traffic of the new site: the false positive rate is 0.23 to 0.81, and up to 0.81 of the core network's own signaling is flagged. In the reverse direction, models trained on $\mathcal{D}_B$ reach 0.61 to 0.75 on $\mathcal{D}_A$ and 0.55 to 0.81 on $\mathcal{D}_C$.

C. WHAT THE GAP CONSISTS OF, AND ADAPTATION

Byte accounting and separability. Removing the Ethernet header changed what the shift diagnostics say. A domain classifier (histogram gradient boosting, five-fold cross-validation) tells $\mathcal{D}_A$ from $\mathcal{D}_B$ flows with a balanced accuracy of 0.997 on the 18 shared columns, but 0.935 on the nine exporter-robust columns, against 0.997 before the correction: the uncorrected header had made the mean packet sizes of the two corpora almost perfectly separable. The robust columns still shift as much as the others ($\bar{W}_1$ of 0.30 against 0.29 after a quantile map fitted on source data; Fig. 4). The correction did not shrink this shift: most $\mathcal{D}_B$ flows are UDP and most $\mathcal{D}_A$ flows TCP, and a 28-byte UDP packet now lies where $\mathcal{D}_A$ has few packets, whereas the uncorrected 42 bytes had placed it among the 40- to 44-byte TCP packets of $\mathcal{D}_A$. The remaining shift is a difference in traffic composition, which no byte accounting removes.

A common exporter. The raw captures of $\mathcal{D}_B$ are public, so we re-extracted them with Zeek (version 8.0.10), the exporter

TABLE 4. The 5G-NIDD authors' pipeline [9] reproduced and moved one protocol at a time (R0: their protocol). Balanced accuracy on all flows, and for RF also balanced accuracy and false positive rate without the benign-labeled copies. Mean over two repetitions per rung. KNN runs at R0 only, as its prediction dominated the runtime on later rungs.

			Balanced accuracy, all flows					RF, w/o copies	
Rung	Split	Features	DT	RF	KNN	NB	MLP	BA	FPR
R0	Random 70/30	Published 10	0.9995	0.9996	0.9988	0.9310	0.9986	0.9997	0.0003
R1	Random 70/30	No `Seq`/`Offset`	0.7059	0.7059	–	0.6814	0.7042	0.9999	0.0003
R2	Capture file	Published 10	0.8594	0.9226	–	0.7501	0.8669	0.9578	0.0018
R2	Capture file	No `Seq`/`Offset`	0.7988	0.7996	–	0.7593	0.8004	0.8958	0.0013
R3	Base station	Published 10	0.7619	0.8949	–	0.6598	0.8577	0.9354	0.0009
R3	Base station	No `Seq`/`Offset`	0.6557	0.6555	–	0.6271	0.6919	0.8284	0.0007

TABLE 5. Transfer from $\mathcal{D}_A$ to $\mathcal{D}_B$ (5G-NIDD) in the shared space with the byte correction, mean over 20 splits. All flows: published labels. Clean: the benign-labeled flood copies removed. Relabeled: the copies labeled as attack. Δ_{BA}: held-out source minus target balanced accuracy. Reference: models trained on $\mathcal{D}_B$ with each capture file held out once (LOFO), evaluated on the same clean flows; Ref.−transfer is the shortfall of transfer. Brackets: Nadeau–Bengio corrected 95% intervals over the 20 splits (group-count ratio), unadjusted; *: significant after Holm correction across the six architectures. Italic rows: input-blind baselines.

	Source	All flows	Clean		Relabeled		Reference (LOFO), clean	
Model	BA	BA	BA	Δ_{BA} [95% CI]	BA	Δ_{BA} [95% CI]	BA	Ref.−transfer [95% CI]
LR	0.837	0.631	0.613	+0.223* [0.11, 0.33]	0.574	+0.263* [0.15, 0.37]	0.959	+0.345* [0.33, 0.36]
DT	0.822	0.569	0.781	+0.042 [−0.17, 0.25]	0.802	+0.020 [−0.20, 0.24]	0.974	+0.194 [0.00, 0.39]
RF	0.808	0.592	0.818	−0.010 [−0.27, 0.25]	0.836	−0.028 [−0.29, 0.23]	0.975	+0.157 [−0.02, 0.33]
XGB	0.865	0.588	0.858	+0.007 [−0.23, 0.24]	0.886	−0.020 [−0.26, 0.22]	0.975	+0.117 [0.00, 0.23]
HGB	0.867	0.584	0.860	+0.007 [−0.19, 0.20]	0.890	−0.023 [−0.22, 0.17]	0.976	+0.115* [0.07, 0.16]
MLP	0.903	0.521	0.688	+0.215 [−0.05, 0.48]	0.715	+0.188 [−0.08, 0.46]	0.973	+0.285* [0.07, 0.50]
Strat.	0.502	0.500	–	–	–	–	–	–
Maj.	0.500	0.500	–	–	–	–	–	–

TABLE 6. Transfer to the third corpus $\mathcal{D}_C$ (5G core testbed, NFStream flows with Ethernet and GTP-U bytes removed [17]): balanced accuracy on held-out source data and on $\mathcal{D}_C$, target ROC-AUC, recall per attack type, and the share of the core's own signaling flagged (Bkg.), at $\tau = 0.5$. Mean over 20 splits from $\mathcal{D}_A$ and 20 from $\mathcal{D}_B$ (split by capture file).

	Trained on $\mathcal{D}_A$							Trained on $\mathcal{D}_B$	
Model	Src. BA	Tgt. BA	AUC	SYN	ICMP	PFCP	Bkg.	Tgt. BA	AUC
LR	0.837	0.742	0.800	1.00	0.10	0.00	0.27	0.552	0.708
DT	0.822	0.658	0.696	1.00	1.00	0.79	0.67	0.603	0.612
RF	0.808	0.596	0.926	1.00	1.00	0.99	0.81	0.699	0.855
XGB	0.865	0.640	0.892	1.00	1.00	0.72	0.70	0.584	0.765
HGB	0.867	0.698	0.932	1.00	1.00	0.80	0.61	0.726	0.773
MLP	0.903	0.677	0.830	1.00	0.20	0.63	0.47	0.811	0.739

family of $\mathcal{D}_A$, and labeled each flow by the attack host pairs of its capture file, a rule that reproduces 100% of the published labels on the Argus flows. Scored by the same trained models, the clean Zeek flows reach 0.68 to 0.79 for the tree-based architectures, −0.10 to −0.07 relative to the corrected Argus flows, and the domain classifier separates $\mathcal{D}_A$ from the Zeek flows as well as from the Argus flows (0.997 on all columns, 0.937 on the robust ones). Our Zeek configuration may differ from the undocumented one behind $\mathcal{D}_A$ in timeouts or single-packet handling, so this comparison cannot separate traffic composition from exporter configuration.

Feature subset and adaptation without target labels. Table 7 tests whether other inputs or adaptation methods close the gap. Restricting the models to the nine exporter-robust columns raises held-out source balanced accuracy for five architectures but lowers it for LR (0.80 against 0.84). On the clean target flows, XGB and HGB keep their level (0.84 and 0.84), whereas DT and RF fall to 0.62 and 0.60: the columns that an exporter moves least are not enough. None of the three adaptation methods that read unlabeled target flows beats the plain tree-based models. Importance weighting changes little (0.74 to 0.84 for the tree-based models), because its domain classifier separates the corpora so well that the weights concentrate on 2.0% to 6.6% of the source flows in effective sample size. CORAL raises the MLP from 0.69 to 0.81 but pushes the target false positive rate of the tree-based models to 0.77 to 0.99, as in our first analysis. Domain-adversarial training reaches 0.66.

A few labeled target captures. Labeled data from the target helps far more (Fig. 5). Added to the $\mathcal{D}_A$ training data, a single labeled capture file of $\mathcal{D}_B$ raises the balanced accuracy of the five nonlinear architectures on the capture files not drawn to 0.87 to 0.93 (the MLP from 0.71 to 0.93), close to the leave-one-capture-file-out reference, and eight files to 0.93 to 0.95.

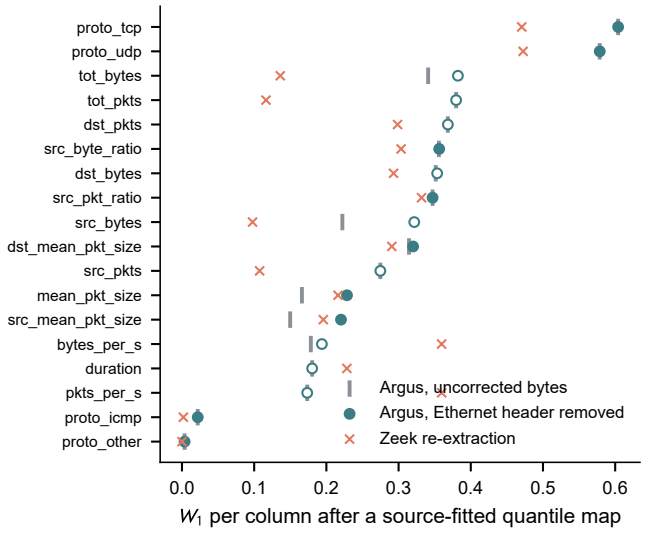


FIGURE 4. Covariate shift between $\mathcal{D}_A$ and $\mathcal{D}_B$ per shared column: first-order Wasserstein distance after a quantile map fitted on $\mathcal{D}_A$ training data. Bars: Argus flows before the byte correction; circles: after it (filled: exporter-robust subset); crosses: our Zeek re-extraction of the 5G-NIDD captures. The correction moves the packet-size columns but does not remove the shift.

TABLE 7. Transfer from $\mathcal{D}_A$ to $\mathcal{D}_B$ under other feature sets and with adaptation, mean over 20 splits, at $\tau = 0.5$. Importance weighting, CORAL, and DANN read unlabeled target flows. Clean: the benign-labeled copies removed; relab.: the copies labeled as attack.

Arm	Model	Src. BA	Target, clean			Target, relab.
			BA	AUC	FPR	BA
Shared space (18)	LR	0.837	0.613	0.796	0.061	0.574
	DT	0.822	0.781	0.783	0.206	0.802
	RF	0.808	0.818	0.879	0.232	0.836
	XGB	0.865	0.858	0.896	0.083	0.886
	HGB	0.867	0.860	0.888	0.064	0.890
	MLP	0.903	0.688	0.806	0.432	0.715
Exporter-robust (9)	LR	0.804	0.615	0.821	0.646	0.632
	DT	0.883	0.616	0.633	0.546	0.647
	RF	0.894	0.599	0.781	0.578	0.630
	XGB	0.925	0.835	0.847	0.076	0.870
	HGB	0.939	0.837	0.860	0.082	0.871
	MLP	0.937	0.651	0.815	0.479	0.681
Importance weighting	LR	0.696	0.624	0.789	0.533	0.630
	DT	0.831	0.782	0.754	0.167	0.802
	RF	0.788	0.812	0.878	0.223	0.830
	XGB	0.831	0.837	0.882	0.103	0.858
	HGB	0.814	0.740	0.863	0.220	0.768
	MLP	0.842	0.684	0.795	0.438	0.707
CORAL	LR	0.836	0.585	0.855	0.741	0.597
	DT	0.818	0.539	0.543	0.794	0.546
	RF	0.805	0.482	0.761	0.985	0.489
	XGB	0.903	0.539	0.736	0.766	0.560
	HGB	0.891	0.521	0.741	0.820	0.536
	MLP	0.932	0.813	0.815	0.177	0.825
DANN	DANN	0.895	0.655	0.814	0.600	0.661

The same capture files alone give only 0.53 to 0.63 for one file, so the source data still carries most of what the detectors know. The alert bound takes more data. With the threshold fixed on source data, the six architectures met all three alert conditions in 2 of 24 draws with one labeled file and in 14 of 24 with eight; with eight files, XGB and the MLP met them in every draw, at 21 and 30 false alerts per hour on average and a recall of about 0.74.

Size of the source sample. Repeating the transfer with 150,000, 600,000, and all 1,640,182 source flows (five splits, five architectures) changed clean target balanced accuracy by at most 0.08 against 300,000 flows, in no consistent direction, which is within the variation between splits.

D. ALERT BURDEN AND THE DEPLOYABILITY TEST

Alert volumes. At the default threshold of 0.5, the transferred detectors raise many false alerts. On the clean flows of $\mathcal{D}_B$, their false positive rate is 0.061 to 0.432, that is, 14,673 to 103,665 false alerts per hour at our declared rate, for an operational precision of 0.0037 to 0.0241 at $\pi = 0.002$. On all flows, where the copies count as benign, the false positive rate reaches 0.767.

The test. Table 8 applies the two-term test of Section III-C. No detector passes in any label view. On all flows, five of the six detectors fail and one is inconclusive. On the clean flows, the generalization term fails for LR and is inconclusive for the other five architectures: their intervals include losses larger than δ, so neither a pass nor a failure is established. The alert term fails for two architectures and is inconclusive for four. Its intervals are wide because the threshold fixed on the source data of each split lands differently on the target: across the 20 splits, one architecture can range from no alerts, and no detections, to tens of thousands of false alerts per hour. Counted split by split, HGB meets all three alert conditions in 12 of the 20 splits, the MLP in 4, XGB in 2, and the other architectures in none. Averaged over the splits, HGB raises 352 false alerts per hour at a recall of 0.51 and an operational precision of 0.41, more alerts than our bound allows. Relabeling the copies gives the same verdicts except for RF, whose alert term becomes inconclusive.

An optimistic bound. A threshold searched on the target shows what the ranking itself allows. With the best such threshold, HGB reaches an operational precision of 0.91 at 25 false alerts per hour on the clean flows, and XGB 0.57 at 181, both within our bounds; two architectures meet the bounds in this way, and none does on all flows, where the best operational precision at a recall of at least 0.5 is 0.005. On consistent labels the best transferred detectors therefore rank the attacks of $\mathcal{D}_B$ well enough for our bounds, but the threshold that would realize this cannot be chosen from source data. In our experiments the threshold, not only the model, had to be set with labeled data from the target.

Sensitivity to the bounds. The bounds are our own, and Fig. 6 shows how far each detector lies from them. On all flows no threshold comes close: even the threshold searched on the target leaves every architecture with at least 54,814 false alerts per hour at a recall of 0.5. On consistent labels the verdict depends on the threshold rule more than on the bounds. At the threshold fixed on source data, the recall interval of every architecture extends below $r_{\min} = 0.5$, so no change of $A_{\max}$ or ρ alone would make the alert term pass, and the generalization term would remain inconclusive unless

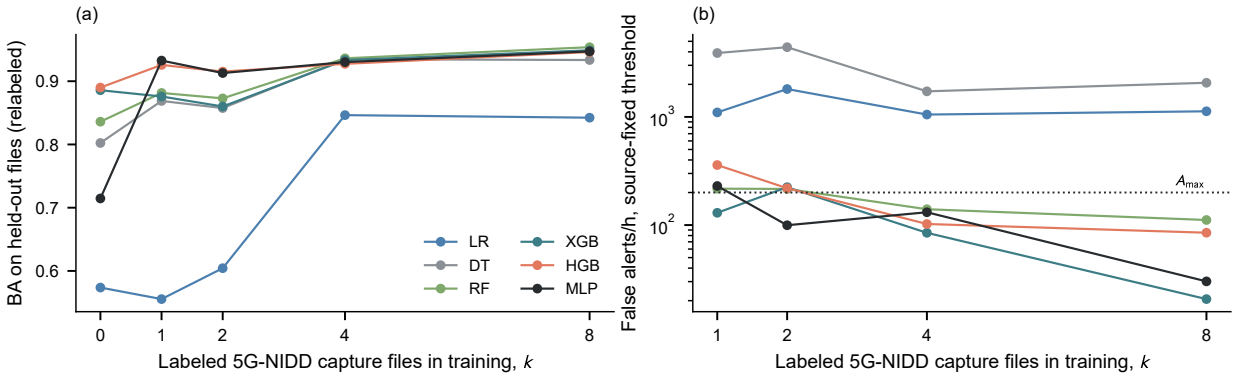


FIGURE 5. Label-efficient arm: k labeled capture files of $\mathcal{D}_B$ added to the $\mathcal{D}_A$ training data (four random draws per k; $k = 0$ is the plain transfer of the 20 splits), evaluated on the capture files not drawn, with the copies relabeled as attack. (a) Balanced accuracy; (b) false alerts per hour at the threshold fixed on source data, mean over draws, with the bound $A_{\max}$.

TABLE 8. Two-term deployability test (Section III-C) for transfer from $\mathcal{D}_A$ to $\mathcal{D}_B$, 20 splits. Generalization: Δ_{BA} with its Nadeau–Bengio corrected 95% interval; pass if the upper bound is at most $\delta = 0.10$, fail if the lower bound exceeds it. Alert burden at the threshold fixed on each split's held-out source data (recall at least 0.5 there): mean recall, false alerts per hour at $\lambda_b = 240{,}000$ benign flows per hour, and operational precision at $\pi = 0.002$, with corrected intervals; pass if all three intervals lie on the passing side of $r_{\min} = 0.5$, $A_{\max} = 200$, and $\rho = 0.1$, fail if any lies on the failing side. Splits: splits in which all three alert conditions hold. ✓: pass, ×: fail, ?: inconclusive; $\mathcal{P}$: verdict. The bounds are our own, not operator requirements.

		Generalization		Alert burden, threshold fixed on source data					
View	Model	Δ_{BA} [95% CI]		Recall [95% CI]	Alerts/h [95% CI]	PPV [95% CI]	Splits		$\mathcal{P}$
All flows	LR	+0.21 [0.10, 0.31]	?	0.04 [0.00, 0.09]	570 [0, 1,832]	0.032 [0.000, 1.000]	0/20	×	×
	DT	+0.25 [0.06, 0.45]	?	0.70 [0.41, 0.98]	146,512 [88,607, 204,416]	0.002 [0.001, 0.005]	0/20	×	×
	RF	+0.22 [−0.02, 0.46]	?	0.14 [0.00, 0.47]	31,967 [0, 107,201]	0.002 [0.000, 1.000]	0/20	×	×
	XGB	+0.28 [0.07, 0.48]	?	0.19 [0.00, 0.58]	45,441 [0, 137,454]	0.002 [0.000, 1.000]	0/20	?	?
	HGB	+0.28 [0.10, 0.47]	?	0.51 [0.16, 0.86]	113,355 [38,968, 187,742]	0.002 [0.000, 0.011]	0/20	×	×
	MLP	+0.38 [0.22, 0.55]	×	0.39 [0.00, 0.84]	85,583 [0, 189,405]	0.002 [0.000, 1.000]	0/20	?	×
Clean	LR	+0.22 [0.11, 0.33]	×	0.04 [0.00, 0.09]	1,387 [0, 4,460]	0.014 [0.000, 1.000]	0/20	×	×
	DT	+0.04 [−0.17, 0.25]	?	0.70 [0.41, 0.98]	46,906 [0, 127,390]	0.007 [0.002, 1.000]	0/20	?	?
	RF	−0.01 [−0.27, 0.25]	?	0.14 [0.00, 0.47]	101 [0, 511]	0.406 [0.000, 1.000]	0/20	×	×
	XGB	+0.01 [−0.23, 0.24]	?	0.19 [0.00, 0.58]	41 [0, 259]	0.691 [0.000, 1.000]	2/20	?	?
	HGB	+0.01 [−0.19, 0.20]	?	0.51 [0.16, 0.86]	352 [0, 2,845]	0.412 [0.027, 1.000]	12/20	?	?
	MLP	+0.21 [−0.05, 0.48]	?	0.39 [0.00, 0.84]	1,762 [0, 7,043]	0.096 [0.000, 1.000]	4/20	?	?
Relabeled	LR	+0.26 [0.15, 0.37]	×	0.03 [0.00, 0.06]	1,387 [0, 4,460]	0.010 [0.000, 1.000]	0/20	×	×
	DT	+0.02 [−0.20, 0.24]	?	0.75 [0.45, 1.00]	46,906 [0, 127,390]	0.008 [0.002, 1.000]	0/20	?	?
	RF	−0.03 [−0.29, 0.23]	?	0.17 [0.00, 0.55]	101 [0, 511]	0.442 [0.000, 1.000]	0/20	?	?
	XGB	−0.02 [−0.26, 0.22]	?	0.23 [0.00, 0.69]	41 [0, 259]	0.728 [0.000, 1.000]	2/20	?	?
	HGB	−0.02 [−0.22, 0.17]	?	0.59 [0.20, 0.99]	352 [0, 2,845]	0.448 [0.032, 1.000]	12/20	?	?
	MLP	+0.19 [−0.08, 0.46]	?	0.45 [0.00, 0.98]	1,762 [0, 7,043]	0.109 [0.000, 1.000]	4/20	?	?

δ were raised to 0.20, the smallest upper bound of Δ_{BA} on these flows.

E. RADIO-LAYER PROTOCOL SENSITIVITY

Table 9 compares a random and a group-disjoint split on the radio layer of $\mathcal{D}_A$, with the 30 capture sessions or the 42 experiment runs as groups; a third protocol, which keeps every attack category on both sides, gives the same picture (repository). Grouped by sessions, a random split raises macro-F_1 over a group-disjoint split by 0.09 to 0.17 across the six architectures, 0.13 on average. This gain is not statistically significant: the corrected interval of the average gain is −0.03 to 0.30 ($p = 0.11$), and no architecture's gain is significant after Holm correction. Grouped by runs, the average gain is 0.15, with a corrected interval of −0.02 to 0.33 ($p = 0.08$), and again no architecture's gain is significant after Holm correction. The input-blind baselines gain nothing under either grouping, so class composition does not move macro-F_1 between protocols. A model-free probe is consistent with recognition of capture groups: under a random split, 68% of the test windows have their nearest training window in the same session, although windows share no records, and a one-nearest-neighbor lookup reaches a macro-F_1 of 0.958, against 0.814 under a session-disjoint split. The probe shows that recognizing the group is available to a model, not that the trained models rely on it.

The radio layer cannot support operating figures. It holds 661 benign windows, about 3 hours of data, recorded mostly with one UE and, for nine of the ten benign sessions, six weeks before the first attack session. When each benign session is held out alone (Fig. 7), the false positive rate averaged over the six architectures is at most 0.07 for six sessions and 0.81 to 0.98 for sessions 4, 8 and 29, and a bootstrap over splits and sessions puts the pooled false positive rate between 0.01 and 0.65 (lowest and highest interval bound across architectures). A false positive rate this dependent on

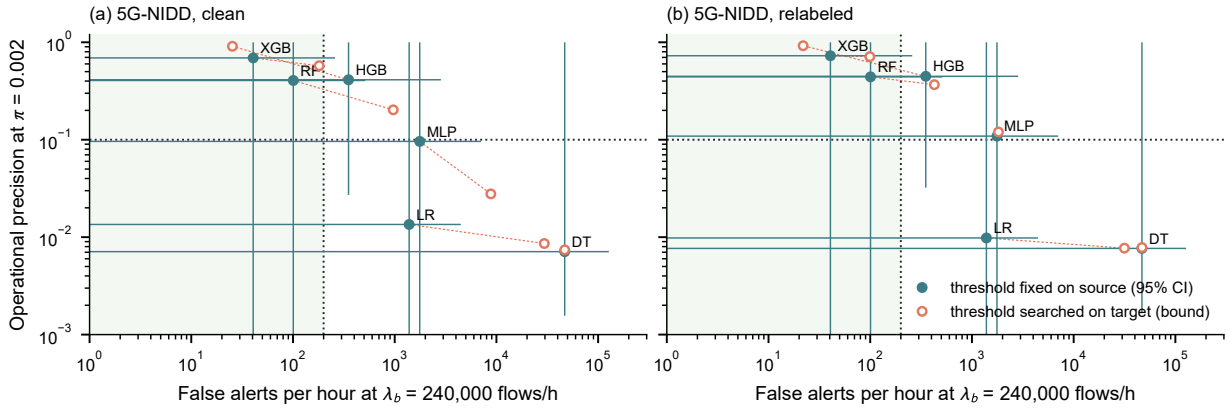


FIGURE 6. Where the transferred detectors lie relative to the alert bounds on $\mathcal{D}_B$: false alerts per hour against operational precision, mean over 20 splits, at the threshold fixed on source data (filled, with corrected 95% intervals) and at the best threshold searched on the target (open, an optimistic bound), for (a) clean flows and (b) copies relabeled. The shaded region meets $A_{\max} = 200$ false alerts per hour; dotted lines mark $A_{\max}$ and $\rho = 0.1$. Both bounds are our own choices; another operator would move the lines.

TABLE 9. Radio-layer protocol sensitivity on $\mathcal{D}_A$: macro-F_1 under a random (R) and a group-disjoint (GD) split, with sessions (30 groups) or runs (42 groups) as the group, and the gain of the random split with its Nadeau–Bengio corrected 95% interval (group counts), over 20 splits with two model seeds each. Intervals are unadjusted; *: significant after Holm correction across the six architectures.

	30 sessions			42 runs		
Model	R	GD	Gain [95% CI]	R	GD	Gain [95% CI]
LR	0.836	0.744	+0.093 [−0.08, 0.26]	0.834	0.738	+0.096 [−0.08, 0.28]
DT	0.950	0.782	+0.168 [−0.01, 0.35]	0.945	0.752	+0.193 [0.02, 0.36]
RF	0.970	0.853	+0.117 [−0.07, 0.30]	0.970	0.816	+0.154 [−0.04, 0.35]
XGB	0.977	0.836	+0.141 [−0.03, 0.32]	0.979	0.813	+0.166 [−0.02, 0.35]
HGB	0.982	0.844	+0.138 [−0.03, 0.31]	0.983	0.816	+0.167 [−0.02, 0.35]
MLP	0.954	0.828	+0.126 [−0.04, 0.29]	0.955	0.814	+0.141 [−0.03, 0.31]
Strat.	0.498	0.507	−0.009 [−0.02, 0.00]	0.497	0.507	−0.010 [−0.02, 0.00]
Maj.	0.431	0.434	−0.002 [−0.02, 0.02]	0.431	0.434	−0.003 [−0.02, 0.01]
Average			+0.131 [−0.03, 0.30]			+0.153 [−0.02, 0.33]

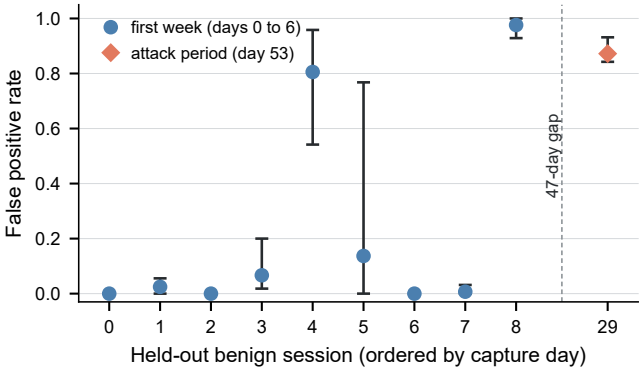


FIGURE 7. False positive rate on each benign radio session of $\mathcal{D}_A$ when that session alone is held out and the models train on the other 29: mean (marker) and range (bar) over the six architectures, sessions in capture order. Within one week of capture, sessions range from no false positives to almost every window flagged, so the radio layer supports no operating figure.

which benign session a detector meets cannot be turned into an alert rate for another network, so we report none for the radio layer.

F. DECISION LATENCY OF WINDOW-LEVEL DETECTORS

We measured latency separately, for window-level detectors only, and it is not part of the test. Our C xApp in FlexRIC v2.0.0 subscribes to E2SM-KPM reports from FlexRIC's emulated gNB over SCTP, with 16 measurement items per UE, scores every UE of a report with ONNX Runtime, answers each report with one E2SM-RC control message, and waits for the acknowledgment. The emulated node reports synthetic values, so the xApp scores held-out radio test windows in their place; the C code reproduced the Python scores on all 589 test windows to within 4×10^{-7}. With a report every 10 ms (5,000 reports per model), the full loop from report to acknowledged control message takes 4.45 to 4.95 ms at the 99th percentile over the six models (Table 10). Delivery and control account for at least 86% of the mean loop time, and inference takes at most 0.229 ms at the 99th percentile, so once a RIC is in the path the model barely matters. At a 1 ms period the loop ran faster (1.42 to 1.69 ms), which fits a host that idles between sparse reports and pays to wake up: these are processing costs of the RIC software path on one host, under WSL2, without CPU isolation and with synthetic measurements over loopback, not the delay of a deployment.

Two limits follow. First, computation is not what limits a window-level detector: it decides once its 16 s window is complete, so a compute tail of a few milliseconds is small next to the detection delay. Second, we have no measured latency for flow-level detectors: their records need a transport to the xApp that our loop lacks and O-RAN does not define, and the cost of a native user-plane exporter is unknown. We therefore make no latency claim for flow detectors.

VI. DISCUSSION

A. IMPLICATIONS FOR DETECTION IN O-RAN

Data path. E2SM-KPM delivers aggregated measurements once per reporting period, not packets. A window-level detector can therefore live inside the RIC, whereas a flow-level

TABLE 10. Decision loop of a window-level detector through FlexRIC with an emulated E2 node on one host, in ms, ranges over models (one trained model per architecture, split seed 101), at a 10 ms reporting period (six models, 5,000 reports each) or a 1 ms period (LR and RF, 20,016 reports each). Terms of Eq. (3): delivery to the xApp (t_{ind}), queueing (t_q), aggregation (t_{feat}) and inference (t_{inf}) for one UE, and control up to acknowledgment (t_{act}). "Loop": KPM header timestamp to acknowledgment, for the first UE of a report or all its UEs.

Term	10 ms, six models		1 ms, LR and RF	
	p_{50}	p_{99}	p_{50}	p_{99}
t_{ind}	1.51–1.86	3.02–3.05	0.51–0.53	0.84–0.98
t_q	0.063–0.071	0.18–0.21	0.015–0.016	0.069
t_{feat}	0.001	0.002–0.003	<0.001	0.001
t_{inf}	0.050–0.12	0.13–0.23	0.014–0.025	0.038–0.050
t_{act}	0.72–0.87	1.40–1.50	0.20–0.21	0.53–0.60
Loop, 1 UE	2.38–2.97	4.36–4.59	0.76–0.79	1.38–1.59
Loop, all	2.48–3.18	4.45–4.95	0.77–0.86	1.42–1.69

detector needs an exporter near the user plane (in the central unit or at the user-plane function) and a way to move its records to the xApp, such as a custom E2 service model or the non-real-time data collection path. This transport adds to t_{ind} in Eq. (3), and nobody has measured it for these detectors, ourselves included.

Detection delay. Computation is a small part of detection time. A window-level decision waits for its 16 s window to end, and a flow-level one for the flow record, which for a long connection means an idle or active timeout. Computational latency should be reported with the KPM reporting period and the flow timeouts.

Acting on alerts. An E2SM-RC action such as isolating or steering a UE hits a falsely accused UE as hard as a guilty one. Without labeled target data, the averaged operating points of our flow detectors met the alert bounds only at thresholds tuned on the target (Section V-D). With thresholds fixed on source data they either missed most attacks, as XGB and RF did with 41 and 101 false alerts per hour at a recall of 0.19 and 0.14, or raised hundreds to tens of thousands of false alerts per hour at our declared rate. Alerts of this quality can inform an operator, or a policy set through the A1 interface of the non-real-time RIC, but should not drive automatic control. For such detectors the 10 ms budget matters less than detection quality: latency binds only for a detector whose alerts are good enough to act on automatically.

Admission and calibration on site. The two-term test could serve as an admission check at a new site, given labeled traffic from it. Our label-efficient arm (Section V-C) suggests that a few labeled captures from the site are worth more than adaptation without labels, and the comparison of fixed and tuned thresholds (Section V-D) that the operating threshold, not only the model, has to be checked on site data.

B. RELATION TO REPORTED RESULTS

The 5G-NIDD results we cite [9], [10] are in-distribution scores, and under the same split ours are as high: 99.87% to 99.96% accuracy for the reproduced pipeline, and 0.987 to 0.990 balanced accuracy inside $\mathcal{D}_C$. Setting such numbers beside transfer results compares protocols, not detectors. In our experiments a held-out score under a random split did not predict performance on another corpus: on 5G-NIDD it rested on record-position fields, the published labels hid the attack traffic of the other base station, and a byte-accounting difference between exporters entered every byte column of the shared space.

VII. LIMITATIONS

Scope of the corpora. Only NetsLab-5GORAN-IDD is an O-RAN corpus. 5G-NIDD comes from a 5G test network and the third corpus from a 5G core in Docker, with three capture files, only flood attacks and one control-plane attack, too few for cluster resampling. Three corpora show that the transfer gap recurs, not how it varies across sites. We found no other O-RAN corpus with paired radio telemetry, so the radio detectors have no transfer target, and no detector is evaluated on all criteria.

Detectors. The six architectures are untuned shallow models with hyperparameters fixed in advance, and we tested three adaptation methods, one per family. Tuned or larger models, other adaptation methods, or detectors built for a specific deployment may transfer differently; our conclusions concern the detectors we tested.

Flow exporters. Zeek, Argus, and NFStream produced the three corpora. We removed the link-layer and tunnel bytes that two of them count, but other exporter differences remain: where a flow ends, and the handling of single-packet flows. Re-extracting 5G-NIDD with Zeek did not reduce the gap (Section V-C), but the Zeek version and configuration behind NetsLab-5GORAN-IDD are undocumented and our Zeek flows take their labels through the host pairs of each capture, so this comparison cannot separate traffic composition from exporter configuration.

Target labels. The packet-level evidence shows that station 1's UDP-flood file contains station 2's flood, but how the file was produced is not documented, and the relabeled view rests on our reading that these flows are attacks. We therefore report every 5G-NIDD result on the published labels, without the copies, and relabeled.

Radio layer. The radio layer has 30 sessions, each benign or from one attack category, so session and category are confounded, and nine of the ten benign sessions were recorded six weeks before the attacks, mostly with one UE. The protocol comparison detects only large effects, and the radio false positive rate depends so much on the benign session that we report no radio operating figure.

Latency. All latency figures come from one workstation without CPU isolation, and the RIC loop used an emulated E2 node with synthetic measurements over loopback SCTP. It measures the processing cost of the RIC software path for window-level detectors, not the delay of a deployment, and no flow detector was timed in the loop.

Operational parameters and the test. The prevalence $\pi = 0.002$, the rate λ_b, and every bound of the test are our own declarations, not operator requirements. Only π enters operational precision, and alert counts scale linearly with λ_b.

An operator may weigh recall, precision, and alert volume differently, and the test ignores detection delay and the cost of a missed attack.

Statistics. The transfer statistics vary the training set over 20 splits and keep the target fixed, so they miss the variability across capture files and other target corpora; a bootstrap over the capture files, reported in the repository, suggests that it is large. The source splits differ much in which addresses they hold out, so the corrected intervals are wide, which is why many terms of the test are inconclusive rather than decided.

VIII. REPRODUCIBILITY AND ETHICS

Code, configuration, seeded splits, per-split results, the corrected 5G-NIDD labels, and the generators for every table, figure, and in-text number are at https://github.com/adeliusa486/oran-ids-deployability, where one script regenerates every number from the results or re-runs each experiment from the SHA-256-verified raw corpora (`REPRODUCE.md`). $\mathcal{D}_A$ [15] and $\mathcal{D}_B$ [16], including the release with all fields and the raw captures, are distributed by their authors under CC-BY-4.0. $\mathcal{D}_C$ is available from its authors' repository [17], which states no license, so we do not redistribute it. All traffic comes from testbed devices and test networks, and no personal data was processed. Showing where detectors fail carries a modest dual-use risk, which we judge smaller than the benefit to operators of knowing that reported accuracy need not carry over to their networks.

IX. CONCLUSION AND REPORTING RECOMMENDATIONS

A held-out score on one corpus said little about how the flow detectors we tested behave on traffic from another network. In 5G-NIDD, 59% of the benign flows are the UDP flood of the other base station, recorded packet for packet in station 1's capture, and a published 99.9% accuracy rests on record-position fields that a deployed detector never sees. Once the labels and the byte accounting of the exporters are corrected, tree-based detectors trained on NetsLab-5GORAN-IDD score as well on 5G-NIDD as on held-out source data, but 0.12 to 0.19 below references trained on 5G-NIDD and tested on the same flows; on the 5G core testbed they catch the flood they were trained on and flag much benign traffic. At a declared prevalence of 0.002 and bounds of our own, no detector passed our two-term test. The best detectors rank the attacks of 5G-NIDD well enough for the bounds, but a threshold fixed on source data did not carry over, and none of three adaptation methods without target labels closed the gap, whereas one labeled target capture closed most of it. On the radio layer, the random-split gain was not significant under either grouping, and the latency we measured for window-level detectors in a FlexRIC loop with an emulated E2 node (99th percentile at most 4.95 ms) is a processing cost on one host, not a conformance result; flow detectors were not timed. We recommend that studies of 5G flow detectors report at least:

1) performance on an independently collected corpus in a prevalence-invariant metric, beside input-blind baselines and a reference trained on that corpus and tested on the same flows;
2) the split protocol and group key, and intervals that account for overlapping splits and clustered samples;
3) how each flow exporter counts bytes, and how the shared feature space removes the differences;
4) evidence that the labels are a function of the features used, such as the share of identical records under opposite labels, and that no record-position or identifier field is a feature;
5) the false positive rate with its unit, the alert rate and operational precision at a stated base rate, and whether the operating threshold was chosen with target labels;
6) for a detector meant for the RIC, the latency from indication to control acknowledgment, with stages, implementation, and host, measured with a real E2 node before any conformance claim.

REFERENCES

[1] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 1376–1411, 2023. doi: 10.1109/COMST.2023.3239220

[2] A. S. Abdalla, P. S. Upadhyaya, V. K. Shah, and V. Marojevic, "Toward next generation open radio access networks: What O-RAN can and cannot do!" *IEEE Network*, vol. 36, no. 6, pp. 206–213, 2022. doi: 10.1109/MNET.108.2100659

[3] S. Niknam, A. Roy, H. S. Dhillon, S. Singh, R. Banerji, J. H. Reed, N. Saxena, and S. Yoon, "Intelligent O-RAN for beyond 5G and 6G wireless networks," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, 2022, pp. 215–220. doi: 10.1109/GCWkshps56602.2022.10008676

[4] L. Bonati, S. D'Oro, M. Polese, S. Basagni, and T. Melodia, "Intelligence and learning in O-RAN for data-driven NextG cellular networks," *IEEE Communications Magazine*, vol. 59, no. 10, pp. 21–27, 2021. doi: 10.1109/MCOM.101.2001120

[5] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Seaman, N. Sullivan, K. Thomas, and Y. Zhou, "Understanding the Mirai botnet," in *Proc. 26th USENIX Security Symposium*, 2017, pp. 1093–1110. [Online]. Available: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis

[6] C. Kolias, G. Kambourakis, A. Stavrou, and J. Voas, "DDoS in the IoT: Mirai and other botnets," *Computer*, vol. 50, no. 7, pp. 80–84, 2017. doi: 10.1109/MC.2017.201

[7] H. Wen, P. Porras, V. Yegneswaran, A. Gehani, and Z. Lin, "5G-Spector: An O-RAN compliant layer-3 cellular attack detection service," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2024. doi: 10.14722/ndss.2024.24527

[8] A. Scalingi, S. D'Oro, F. Restuccia, T. Melodia, and D. Giustiniano, "Det-RAN: Data-driven cross-layer real-time attack detection in 5G open RANs," in *Proc. IEEE Conf. on Computer Communications (INFOCOM)*, 2024, pp. 41–50. doi: 10.1109/INFOCOM52122.2024.10621223

[9] S. Samarakoon, Y. Siriwardhana, P. Porambage, M. Liyanage, S.-Y. Chang, J. Kim, J. Kim, and M. Ylianttila, "5G-NIDD: A comprehensive network intrusion detection dataset generated over 5G wireless network," arXiv:2212.01298, 2022. [Online]. Available: https://arxiv.org/abs/2212.01298

[10] L. Ilias, G. Doukas, V. Lamprou, C. Ntanos, and D. Askounis, "Convolutional neural networks and mixture of experts for intrusion detection in 5G networks and beyond," arXiv:2412.03483v2, 2025. [Online]. Available: https://arxiv.org/abs/2412.03483v2

[11] S. Layeghy and M. Portmann, "Explainable cross-domain evaluation of ML-based network intrusion detection systems," *Computers and Electrical Engineering*, vol. 108, p. 108692, 2023. doi: 10.1016/j.compeleceng.2023.108692

[12] S. Layeghy, M. Gallagher, and M. Portmann, "Benchmarking the benchmark: Comparing synthetic and real-world network IDS datasets," *Journal*

of Information Security and Applications, vol. 80, p. 103689, 2024. doi: 10.1016/j.jisa.2023.103689

[13] S. Axelsson, "The base-rate fallacy and the difficulty of intrusion detection," *ACM Transactions on Information and System Security*, vol. 3, no. 3, pp. 186–205, 2000. doi: 10.1145/357830.357849

[14] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symposium on Security and Privacy*, 2010, pp. 305–316. doi: 10.1109/SP.2010.25

[15] F. A. Zadeh, A. Civciss, V. Ravihansa, C. Sandeepa, and M. Liyanage, "Descriptor: 5G open radio access network multi-modal intrusion detection dataset (NetsLab-5GORAN-IDD)," *IEEE Data Descriptions*, vol. 2, pp. 348–357, 2025. doi: 10.1109/IEEEDATA.2025.3614167

[16] S. Samarakoon, Y. Siriwardhana, P. Porambage, M. Liyanage, S.-Y. Chang, J. Kim, J. Kim, and M. Ylianttila, "5G-NIDD: A comprehensive network intrusion detection dataset generated over 5G wireless network," IEEE DataPort, 2022. doi: 10.21227/xtep-hv36

[17] B. Nugraha, M. Hajizadeh, T. Niehoff, A. V. Jnanashree, T. V. Phan, D. Triantafyllopoulou, O. Krause, M. Mieth, K. Moessner, and T. Bauschert, "A comprehensive 5G dataset for control and data plane security and resource management," in *Proc. IEEE International Conference on Cyber Security and Resilience (CSR)*, 2025, pp. 326–333. doi: 10.1109/CSR64739.2025.11130023

[18] A. Abraheem and A. Edhirig, "Bidirectional cross-dataset generalization and label-efficient adaptation for 5G network intrusion detection," *Wadi Alshatti University Journal of Pure and Applied Sciences*, vol. 4, no. 2, Aug. 2026. [Online]. Available: https://waujpas.com/index.php/journal/article/view/509

[19] L. Obiuwevwi, K. J. Rechowicz, S. Jayarathna, S. H. Bouk, F. Afrin, C. N. Barati, N. Moghim, V. Nannou, M. E. Ur Rahman, and S. Shetty, "Enabling real-time AI in O-RAN: Deploying and measuring AI inside a near-RT RIC xApp," in *Proc. 35th Int. Conf. on Computer Communications and Networks (ICCCN)*, Jul. 2026, pp. 1–9. doi: 10.1109/ICCCN69946.2026.11662852

[20] L. Bonati, M. Polese, S. D'Oro, S. Basagni, and T. Melodia, "OpenRAN Gym: AI/ML development, data collection, and testing for O-RAN on PAWR platforms," *Computer Networks*, vol. 220, p. 109502, 2023. doi: 10.1016/j.comnet.2022.109502

[21] L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "SCOPE: An open and softwarized prototyping platform for NextG systems," in *Proc. ACM MobiSys*, 2021, pp. 415–426. doi: 10.1145/3458864.3466863

[22] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "ColO-RAN: Developing machine learning-based xApps for open RAN closed-loop control on programmable experimental platforms," *IEEE Transactions on Mobile Computing*, vol. 22, no. 10, pp. 5787–5800, 2023. doi: 10.1109/TMC.2022.3188013

[23] R. Schmidt, M. Irazabal, and N. Nikaein, "FlexRIC: An SDK for next-generation SD-RANs," in *Proc. ACM CoNEXT*, 2021, pp. 411–425. doi: 10.1145/3485983.3494870

[24] N. Nikaein, M. K. Marina, S. Manickam, A. Dawson, R. Knopp, and C. Bonnet, "OpenAirInterface: A flexible platform for 5G research," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 5, pp. 33–38, 2014. doi: 10.1145/2677046.2677053

[25] I. Gomez-Miguelez, A. Garcia-Saavedra, P. D. Sutton, P. Serrano, C. Cano, and D. J. Leith, "srsLTE: An open-source platform for LTE evolution and experimentation," in *Proc. ACM WiNTECH*, 2016, pp. 25–32. doi: 10.1145/2980159.2980163

[26] ETSI, "Publicly available specification (PAS); O-RAN security threat modeling and risk assessment (O-RAN.WG11.Threat-Modeling.O-R003-v03.00)," European Telecommunications Standards Institute, ETSI TR 104 106 V3.0.0, Jun. 2025. [Online]. Available: https://www.etsi.org/deliver/etsi_tr/104100_104199/104106/03.00.00_60/tr_104106v030000p.pdf

[27] M. Liyanage, A. Braeken, S. Shahabuddin, and P. Ranaweera, "Open RAN security: Challenges and opportunities," *Journal of Network and Computer Applications*, vol. 214, p. 103621, 2023. doi: 10.1016/j.jnca.2023.103621

[28] D. Mimran, R. Bitton, Y. Kfir, E. Klevansky, O. Brodt, H. Lehmann, Y. Elovici, and A. Shabtai, "Security of open radio access networks," *Computers & Security*, vol. 122, p. 102890, 2022. doi: 10.1016/j.cose.2022.102890

[29] J. Groen, S. D'Oro, U. Demir, L. Bonati, M. Polese, T. Melodia, and K. R. Chowdhury, "Implementing and evaluating security in O-RAN: Interfaces, intelligence, and platforms," *IEEE Network*, vol. 39, no. 1, pp. 227–234, 2025. doi: 10.1109/MNET.2024.3434419

[30] H. Fard, I. Komarov, and G. Wunder, "Cross-layer intrusion detection in 5G O-RAN: Gains and limits of fusing radio telemetry with network flow records," arXiv:2606.22450, 2026. [Online]. Available: https://arxiv.org/abs/2606.22450

[31] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019. doi: 10.1016/j.future.2019.05.041

[32] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets," *Sustainable Cities and Society*, vol. 72, p. 102994, 2021. doi: 10.1016/j.scs.2021.102994

[33] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40 281–40 306, 2022. doi: 10.1109/ACCESS.2022.3165809

[34] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023. doi: 10.3390/s23135941

[35] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116. doi: 10.5220/0006639801080116

[36] Y. Siriwardhana, S. Samarakoon, P. Porambage, M. Liyanage, S.-Y. Chang, J. Kim, J. Kim, and M. Ylianttila, "Descriptor: 5G wireless network intrusion detection dataset (5G-NIDD)," *IEEE Data Descriptions*, vol. 2, pp. 358–369, 2025. doi: 10.1109/IEEEDATA.2025.3592888

[37] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, "NetFlow datasets for machine learning-based network intrusion detection systems," in *Big Data Technologies and Applications (BDTA/WiCON), LNICST*, vol. 371, 2021, pp. 117–135. doi: 10.1007/978-3-030-72802-1_9

[38] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," *Mobile Networks and Applications*, vol. 27, no. 1, pp. 357–370, 2022. doi: 10.1007/s11036-021-01843-0

[39] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12. doi: 10.1109/SPW53761.2021.00009

[40] R. Flood, G. Engelen, D. Aspinall, and L. Desmet, "Bad design smells in benchmark NIDS datasets," in *Proc. IEEE European Symposium on Security and Privacy (EuroS&P)*, 2024, pp. 658–675. doi: 10.1109/EuroSP60621.2024.00042

[41] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symposium*, 2022, pp. 3971–3988. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/arp

[42] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023. doi: 10.1016/j.patter.2023.100804

[43] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *Proc. 28th USENIX Security Symposium*, 2019, pp. 729–746.

[44] G. Apruzzese, P. Laskov, E. Montes de Oca, W. Mallouli, L. Brdalo Rapa, A. V. Grammatopoulos, and F. Di Franco, "The role of machine learning in cybersecurity," *Digital Threats: Research and Practice*, vol. 4, no. 1, pp. 1–38, 2023. doi: 10.1145/3545574

[45] D. Kus, E. Wagner, J. Pennekamp, K. Wolsing, I. B. Fink, M. Dahlmanns, K. Wehrle, and M. Henze, "A false sense of security? Revisiting the state of machine learning-based industrial intrusion detection," in *Proc. ACM Cyber-Physical System Security Workshop (CPSS)*, 2022, pp. 73–84. doi: 10.1145/3494107.3522773

[46] L. D'hooge, T. Wauters, B. Volckaert, and F. De Turck, "Inter-dataset generalization strength of supervised machine learning methods for intrusion detection," *Journal of Information Security and Applications*, vol. 54, p. 102564, 2020. doi: 10.1016/j.jisa.2020.102564

[47] H. Shimodaira, "Improving predictive inference under covariate shift by weighting the log-likelihood function," *Journal of Statistical Planning and Inference*, vol. 90, no. 2, pp. 227–244, 2000. doi: 10.1016/S0378-3758(00)00115-4

[48] S. Bickel, M. Brückner, and T. Scheffer, "Discriminative learning under covariate shift," *Journal of Machine Learning Research*, vol. 10, pp. 2137–2155, 2009.

[49] B. Sun, J. Feng, and K. Saenko, "Return of frustratingly easy domain adaptation," in *Proc. 30th AAAI Conf. on Artificial Intelligence*, 2016, pp. 2058–2065. doi: 10.1609/aaai.v30i1.10306
[50] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, and V. Lempitsky, "Domain-adversarial training of neural networks," *Journal of Machine Learning Research*, vol. 17, no. 59, pp. 1–35, 2016.
[51] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003. doi: 10.1023/A:1024068626366
[52] ETSI, "Publicly available specification (PAS); E2 interface: General aspects and principles (O-RAN.WG3.E2GAP-R003-v04.01)," European Telecommunications Standards Institute, ETSI TS 104 038 V4.1.0, Oct. 2024. [Online]. Available: https://www.etsi.org/deliver/etsi_ts/104000_104099/104038/04.01.00_60/ts_104038v040100p.pdf
[53] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. doi: 10.1023/A:1010933404324
[54] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. doi: 10.1145/2939672.2939785
[55] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html
[56] C. A. Field and A. H. Welsh, "Bootstrapping clustered data," *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 69, no. 3, pp. 369–390, 2007. doi: 10.1111/j.1467-9868.2007.00593.x
[57] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
[58] X. Jin, L. E. Li, L. Vanbever, and J. Rexford, "SoftCell: Scalable and flexible cellular core network architecture," in *Proc. 9th ACM Conf. on Emerging Networking Experiments and Technologies (CoNEXT)*, 2013, pp. 163–174. doi: 10.1145/2535372.2535377



ADEEL AHMAD (Student Member, IEEE) is currently pursuing the B.S. degree in information technology at the Islamic University of Madinah, Saudi Arabia, where he is a recipient of a Saudi Government-funded scholarship for academic excellence. His research interests include cybersecurity, software-defined networking, machine learning, and cloud computing. He is actively involved in applied machine learning research, including explainable AI for attack detection and privacy-preserving federated learning. He has authored research accepted at the IEEE Vehicular Technology Conference (VTC2026-Fall).



ARSHAD ALI (Senior Member, IEEE) joined the Islamic University of Madinah in December 2014, where he is a full Professor of Data Communication and Networking in the Faculty of Computer and Information Systems. He received the M.Sc. degree in telecommunication technology from Aston University, Birmingham, UK, in 2007, and the Ph.D. degree from Lancaster University, UK, in 2012, after research appointments including the University of Cambridge (2007–2009). With over 18 years of teaching experience, he has taught more than 13 computing courses and mentored over 15 graduate students. His research interests include cybersecurity, wireless sensor networking, and artificial intelligence, and he has published over 90 peer-reviewed articles. He is an ABET Information Technology Program Evaluator.



ERAJ KHAN (Member, IEEE) received the Ph.D. degree in communication security from Lancaster University, UK. He is currently involved in teaching and applied research with the Higher Colleges of Technology, UAE. His current research focuses on the design and evaluation of practical, resource-efficient security solutions for constrained and critical infrastructure environments, and on network security, with emphasis on lightweight security mechanisms for the Internet of Things (IoT) and cyber-physical systems.



GAHANGIR HOSSAIN (Senior Member, IEEE) received the Ph.D. degree in computer engineering from the University of Memphis, TN, USA, in 2014, with a focus on computer science, cognitive science, and information systems. With over 20 years of teaching experience, he has taught more than 18 computing courses and mentored over 20 graduate students, including three Ph.D. candidates. His research interests include cybersecurity, data science, cognitive neuroscience, AI, and recommender systems. He has published over 150 peer-reviewed articles and directs the SCORE Laboratory at the University of North Texas. He serves as an Associate Editor for several journals and is an ABET Computing Program Evaluator.



ALI AKARMA is currently pursuing the B.S. degree in information technology at the Islamic University of Madinah, Saudi Arabia. He previously completed an Associate Degree Program (ADP) in Computer Science at The Islamia University of Bahawalpur, Pakistan. His research focuses on agentic AI systems, trustworthy machine learning, and AI safety, with interests in multi-agent systems, reinforcement learning, retrieval-augmented generation, and AI governance. His work explores the development of reliable, secure, and transparent autonomous AI systems.